최기환

Frontend Engineer

rlghks3004@gmail.com GitHub gihwan-dev.tistory.com

입사 초기에는 통합 인스턴스 모니터링 대시보드의 기획과 디자인, 개발을 혼자 맡았습니다. 입사 2년 차인 지금은 대시보드와 MaxGauge 레거시 전 제품, 사내 React 그리드 라이브러리까지 혼자 관리하고 있습니다. 차세대 MaxGauge 개발에도 기여하며 업무 영역을 넓혀가고 있습니다.

01 **소프트웨어가 계속해서 나아질 수 있는 환경을 설계합니다.** 테스트 코드로 기존 기능을 지속적으로 검증하고 회귀를 방지해, 코드를 바꾼 결과가 실제 개선인지 확인할 수 있도록 합니다.

02 **좋은 개발자는 회사에 돈을 잘 벌어주는 개발자라고 생각합니다.** 업무의 단기적 가치와 장기적 가치, 필요한 리소스를 저울질하며 선택하려고 노력합니다.

03 **귀찮은 일을 가장 싫어합니다.** 번거로운 로컬 환경 설정을 스킬과 도구로 만들어, 작업할 때마다 설정 방법을 찾아가며 준비하던 일을 줄였습니다.

경력

주식회사엑셈 / 프론트엔드 개발자

2024.11 - 현재

exem-table | 사내 React 그리드 라이브러리 2025.06경부터 담당

`React / TypeScript / TanStack Table / Vitest`

기존 컴포넌트의 유지보수를 넘겨받아 API와 렌더링 구조를 재설계하고, 인수 테스트로 기존 동작을 검증할 수 있도록 개선해왔습니다. 현재 사내 5개 제품에서 사용하고 있습니다.

- **기능별 API를 정리하고 상태 소유권을 명확히 구분**
  - 기능마다 설정 방식이 달라 내부 구현을 읽어야 사용법을 알 수 있었습니다. 정렬과 검색 등 같은 기능의 옵션과 콜백을 한곳에 모으고, 제어와 비제어 방식으로 API를 통일했습니다.
  - 화면이 상태를 관리할지 라이브러리에 맡길지 명확히 정하고, 테이블 컴포넌트가 중간에서 따로 보관하던 상태를 없앴습니다.
  - 상태를 여러 곳에 복제해 맞추던 구조가 사라지면서, 동기화를 위해 사용하던 불필요한 useEffect를 제거했습니다.
- **렌더링 제약을 줄이고 가상화를 기본으로 채택**
  - 기존 table 태그는 정해진 중첩 구조와 레이아웃 규칙 때문에 컬럼 리사이즈와 드래그 이동, 가상화를 구현할 때 우회 처리가 필요했습니다.
  - 행과 셀의 배치를 직접 제어할 수 있도록 div 기반으로 바꿨습니다. 필요한 UI를 배치하기 위해 table의 기본 구조를 우회하던 부담을 줄였습니다.
  - 가상화를 옵션으로 두면 같은 기능을 일반 렌더링과 가상화 경로에서 각각 구현하고 검증해야 했습니다. 가상화로 생기는 제약보다 두 경로를 유지하는 비용이 크다고 판단해 기본 동작으로 정했습니다.
- **제품별 요구사항을 수용할 수 있도록 라이브러리의 책임 구분**
  - 제품마다 달라지는 필터 동작과 툴바 및 푸터 UI, DB 지표별 셀 표현과 클릭 동작은 제품 코드에서 정하도록 했습니다.
  - 정렬과 선택, 가상화처럼 제품과 무관하게 재사용할 동작은 exem-table에 두었습니다. 제품별 요구사항을 추가할 때 공통 라이브러리에 제품 전용 분기를 넣지 않도록 경계를 정했습니다.
  - 이 구조에 맞춰 라이브러리를 수정하고 차세대 MaxGauge의 22개 화면을 직접 전환했습니다. 제품 안에서 반복되는 규칙은 공통 컴포넌트로 묶어 화면마다 다시 구현하지 않도록 했습니다.
  - 259개 컬럼 정의를 한곳에 모아, 같은 DB 지표의 이름이나 타입이 바뀌면 각 화면을 찾아 고칠 필요가 없도록 했습니다.

exem-table (계속)

- **기존 인수 테스트를 안전망으로 TanStack Table v9 전환**
  - 상태 하나가 바뀌어도 테이블 전체가 렌더링에 참여했고, 이를 줄이기 위한 비교 함수도 복잡해지고 있었습니다. 필요한 상태만 구독할 수 있는 v9을 활용해 이 구조를 바꾸기로 했습니다.
  - TanStack에 직접 의존하던 코드를 별도 모듈로 모은 뒤 기능별로 전환했습니다. 각 컴포넌트가 필요한 상태만 구독하도록 바꾸고, v9과 중복되던 상태 관리 계층을 제거했습니다.
  - 지속적으로 작성해온 인수 테스트는 Vitest Browser Mode에서 실제 브라우저를 조작하고 화면 결과를 확인하도록 구성했습니다. 이 테스트로 정렬과 검색, 선택, 리사이즈 동작을 확인하며 내부 구조를 바꿨습니다.
  - 10,000행과 8개 컬럼에서 14자 검색어를 입력할 때 누적 React 렌더 시간이 1,927ms에서 61ms로 줄었습니다. 당시 같은 브라우저와 데이터, 조작 조건에서 5회 측정한 중앙값입니다.

관련 글: Tanstack Table V9 마이그레이션 회고

MaxGauge 레거시 제품군 | 유지보수와 개발 절차 개선 `2026.09부터 전담`

`JavaScript / ExtJS / Playwright / Claude Code / Codex`

데이터베이스 성능 모니터링 제품 MFO, MFT, MFA, MFD, MFS의 프론트엔드를 혼자 맡고 있습니다. 고객 요청 분석부터 재현과 구현, 검증, 버전 관리와 패키징까지 담당하고 있습니다.

- **레거시 유지보수 흐름에 맞춘 AI 하네스 구축**
  - 수정이 다른 화면이나 고객사 기능에 영향을 주기 쉬운 레거시를 다루기 위해, 환경 준비부터 구현과 검증, 리뷰까지 이어지는 작업 절차를 하네스로 만들었습니다.
  - 복잡한 로컬 환경 구성을 도구화하고, 실제 환경과 유사한 실행 환경에서 AI가 브라우저를 직접 조작해 작업물을 검증하도록 했습니다.
  - 과거 MR에서 반복된 실수와 사이드 이펙트를 조사해 리뷰 스킬에 반영했습니다. 복제된 코드의 수정 누락이나 비동기 응답 순서 문제처럼 놓치기 쉬운 부분을 점검하도록 했습니다.
  - 테스트와 AI 리뷰를 거친 결과는 시나리오별 화면으로 남기도록 했습니다. 매번 코드를 세세히 읽고 직접 재현하던 검토를 줄이고, 화면에서 의도한 동작을 확인하며 병합할 수 있게 됐습니다.

- **사용자 매뉴얼과 과거 버그 수정 이력을 바탕으로 인수 테스트 도입**
  - MFO의 사용자 매뉴얼을 기준으로 화면에서 사용자가 수행하는 작업과 기대 결과를 인수 테스트로 작성했습니다.
  - 최근 2년간 병합된 버그 수정 MR을 바탕으로, 수정 전에는 실패하고 수정 후에는 통과하는지 확인해 테스트가 실제 결함을 잡는지 검증했습니다.
  - 기능을 수정할 때마다 테스트로 기존 동작을 확인하고 있습니다. 테스트를 작성하는 과정에서 발견한 기존 결함은 별도로 기록했으며 후속 릴리스에서 수정할 예정입니다.

- **제품 버전과 빌드 날짜가 혼재하던 버저닝 기준 정리**
  - 릴리스 담당자마다 버전 부여 방식이 달라, 같은 제품 버전인데 빌드 날짜에 따라 다른 버전처럼 표시되기도 했습니다. 고객사별 변경까지 더해지면 버전만 보고 어떤 소스와 작업이 포함됐는지 알기 어려웠습니다.
  - 제품의 기준 버전과 고객사 식별, 수정 순번의 의미를 정리하고 버전 부여 규칙을 명확히 했습니다.
  - 확정한 버전에 작업 목록과 클라이언트 및 SQL 소스 커밋을 연결해, 어떤 변경으로 만든 패키지인지 추적할 수 있도록 했습니다.

MaxGauge 레거시 (계속)

- **고객 요청을 처리할 기준 브랜치와 수정 전파 규칙 정립**
  - 고객사 브랜치가 없으면 태그나 불명확한 버전 정보를 보고 작업을 시작할 커밋을 사람이 추론해야 했습니다.
  - 공통 개발용 trunk, 지원 버전별 릴리스 브랜치, 고객사별 유지보수 브랜치의 역할을 나눴습니다. 고객 요청은 해당 고객사 브랜치에서 시작하고, 신규 고객은 확인된 릴리스 브랜치에서 시작하도록 기준을 정했습니다.
  - 핫픽스를 어느 지원 버전과 고객사에 반영할지, 고객사에서 수정한 내용을 trunk에 어떻게 반영할지도 규칙으로 정리했습니다. 현재 기존 브랜치를 이 체계로 전환하고 있습니다.

MaxGauge Dashboard | 통합 인스턴스 모니터링 대시보드

입사 초기부터 담당

`Vue에서 React 및 TypeScript로 전환`

여러 DB 인스턴스의 상태와 알람을 한 화면에서 확인하는 제품입니다. 별도 기획자와 디자이너 없이 고객 요구사항을 받아 기획과 디자인, 개발을 맡았습니다. 고객사별로 갈라진 구현을 공통 제품으로 정리하고 사내 디자인 시스템을 활용하기 위해 React로 전환했습니다. 현재 고객사에서 운영 중입니다.

- **고객사별 브랜치에 반복하던 공통 수정 제거**
  - 고객사마다 별도 브랜치에서 기능을 개발해, 공통 버그를 고쳐도 다른 고객사 브랜치에 같은 수정을 반복해서 옮겨야 했습니다.
  - 고객 요청을 공통 제품의 기능으로 추가하고, 고객사별 설정으로 필요한 기능을 켜거나 끄도록 바꿨습니다.
  - 같은 코드에서 고객사별 기능을 제공하게 되면서 공통 버그 수정을 여러 브랜치에 전파하던 작업을 없앴습니다.
- **디자인 시스템을 활용해 화면 개발 속도와 일관성 개선**
  - 혼자 기획과 디자인까지 맡는 상황에서 화면을 개별적으로 만들다 보니 작업마다 스타일과 테마가 달라졌습니다.
  - 사내 React 디자인 시스템의 컴포넌트와 디자인 토큰, 테마 시스템을 적용했습니다.
  - 공통 컴포넌트를 조합해 화면을 만들면서 개발이 빨라졌고, 새 기능에도 같은 디자인 기준과 테마를 적용할 수 있게 됐습니다.
- **TypeScript와 테스트로 배포 전 오류를 확인하는 안전망 마련**
  - TypeScript를 도입해 잘못된 타입 사용을 검사하고, 사용자 조작에 따른 화면 결과는 테스트로 검증하도록 했습니다.
  - 화면 구조나 데이터 처리 코드를 바꿀 때 테스트를 실행해 기존 기능이 유지되는지 확인하고 있습니다.
  - 타입 오류와 테스트로 검증하는 기능의 회귀를 배포 전에 발견할 수 있게 됐습니다.

MaxGauge 차세대 | 데이터베이스 성능 모니터링 제품

`React / TypeScript / TanStack Query / ECharts`

단일 SQL과 DB 객체의 상세 정보를 확인하는 화면, 실시간 커스텀 대시보드의 위젯 생성 폼을 개발했습니다. 화면 전반에서 재사용하는 공통 컴포넌트도 설계하고 구현했으며, 현재는 제품 전반의 기능 수정과 구조 개선에 기여하고 있습니다.

- **새 시각화를 플러그인으로 추가할 수 있도록 위젯 빌더 구성**
  - 차트 미리보기에 데이터 준비와 표시 코드가 섞여 있어 테이블 등 다른 시각화를 추가하기 어려웠습니다.
  - 데이터 준비와 화면 표시의 책임을 분리하고, 시각화별 설정 폼과 검증 규칙, 미리보기를 플러그인 단위로 묶었습니다.
  - 새 차트나 테이블을 추가할 때 해당 플러그인 안에서 관련 코드를 함께 작성하고 수정할 수 있도록 했습니다.
- **사용자가 선택한 SQL부터 보여주도록 조회 순서 변경**
  - 차트에서 선택한 SQL이 목록 뒤쪽에 있으면 앞 페이지를 모두 받은 뒤에야 해당 행을 보여줄 수 있었습니다.
  - 해당 SQL이 포함된 페이지를 먼저 받고, 앞뒤 데이터는 스크롤할 때 조회하도록 바꿨습니다.
  - 추가 조회 중에는 아직 받지 않은 영역만 로딩으로 표시해, 이미 표시한 행은 계속 선택하고 스크롤할 수 있게 했습니다.

MaxGauge 차세대 (계속)

- **드문 대용량 사례를 위한 최적화보다 일반적인 사용 경험을 우선**
  - 차트 리사이즈 중 캔버스를 늘려 보여주고 200ms 뒤 다시 그리던 최적화 때문에 글자와 점이 찌그러졌습니다.
  - 측정에서는 CPU 4배 감속 조건의 21만 점 Scatter 차트에서 끊김을 확인했습니다. 아직 출시 전이고 흔하지 않은 조건이어서, 이 경우를 위해 일반적인 화면의 선명도를 포기하지 않기로 했습니다.
  - 캔버스 확대와 지연 처리를 제거하고 requestAnimationFrame으로 프레임당 한 번 실제 크기에 맞춰 그리도록 바꿨습니다.
- **CI 실행 비용을 분석해 파이프라인 시간 단축**
  - CI가 느려졌지만 큐 대기 중앙값은 1.6초였습니다. 러너를 늘리기 전에 실제 실행 단계에서 시간이 쓰이는 곳을 확인했습니다.
  - Storybook은 미리 빌드한 파일로 제공하고, DOM이 필요 없는 테스트는 Node 환경에서 실행하도록 바꿨습니다. 중복 분석과 리포트 생성도 줄였습니다.
  - 당시 파이프라인 시간 중앙값이 198초에서 94초로 줄었습니다. 변경 전 20회와 후 4회를 비교한 기록이며 재시도와 캐시 재생성은 제외했습니다.

Task Analyzer | 작업 맥락 검색 도구 실험 프로젝트

`Python / TypeScript / MCP`

- 개발에 필요한 요구사항과 관련 코드, 과거 작업 이력을 여러 도구에서 검색할 수 있도록 데이터 관계와 검색 기능을 별도 서버로 분리했습니다.
- MCP나 스킬을 통해 같은 검색 기능을 사용하도록 규격을 정하고 사내 플러그인 마켓에 구성했습니다.
- 회의록에 관련 태스크 링크를 남기거나 개발 중 필요한 작업 맥락을 찾는 데 사용하고 있습니다.

GitLab 슬래시 커맨드 | MR 스레드에서 AI와 협업 실험 프로젝트

`TypeScript / LangGraph`

- MR의 작업 맥락을 개인 PC로 옮겨 설명할 필요가 없도록, 댓글에서 AI를 호출하고 답변과 수정 결과를 같은 스레드에서 검토할 수 있게 했습니다.
- 논의가 섞이지 않도록 대화 이력을 스레드별로 분리했습니다. 함께 작업하는 사람이 같은 맥락에서 AI의 답변을 확인하고 리뷰할 수 있도록 했습니다.

학력

동의대학교 정보통신공학과 / 응용 소프트웨어 공학 부전공 | 2017.03 - 2024.03

최근 글 전체 글 최신 5개 / 2026.10.25 확인

자본주의 1부 - 돈은 빚이다
2026.09.20

지금 테스트 코드가 더더욱 중요한 이유
2026.09.06

스틱 후기
2026.08.29

Tanstack Table V9 마이그레이션 회고
2026.08.26

개발자로 살아남기 - 성장에 대한 고민
2026.08.09